

Liste în Python - Introducere

1. Ce este o listă?

În limbajul Python, o **listă** este o structură de date fundamentală care îți permite să stochezi mai multe elemente într-o singură variabilă. Este precum un container care te lasă să depozitezi mai multe tipuri de obiecte într-o ordine specifică.

Lista:

1	"a"	145	„Ana”	0
---	-----	-----	-------	---

Caracteristici principale:

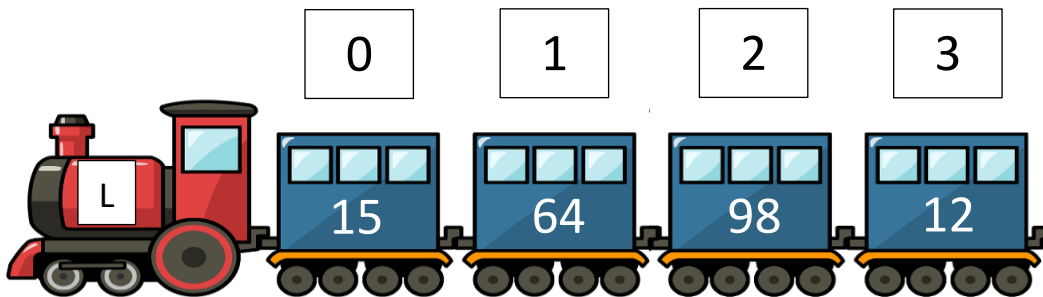
- **Ordonată:** Elementele stau într-o ordine bine stabilită pe care calculatorul o ține minte.
- **Mutabilă (Modificabilă):** Poți să modifice, să adaugi sau să ștergi elemente chiar și după ce lista a fost creată.
- **Flexibilă:** Poate conține date de tipuri diferite în același timp (numere, texte, ba chiar și alte liste!).

2. Crearea unei liste (Sintaxa de bază)

Formula generală: `nume_lista = [element0, element1, element2]`

Exemplu:

```
L = [15, 64, 98, 12] # Am creat o listă cu 4 numere întregi
```



Lista poate fi modelată cu ajutorul acestui tren cu vagoanele numerotate de la 0 la 3.

15 a venit primul la gară și și-a cumpărat bilet, primind loc în vagonul 0.

64 a fost plasat în vagonul 1, iar 98 în vagonul 2.

12 a prins în cele din urmă loc doar în ultimul vagon.

Așadar, controlorul poate găsi orice număr pentru că știe exact în ce vagon se află.

Crearea listelor:

Pentru a defini o listă în Python, valorile pe care dorim să le stocăm (numite și elemente) trebuie încadrate între paranteze pătrate `[ ]` și separate prin virgulă.

```
jocuri = ["Minecraft", "Roblox", "FIFA"]  
print(jocuri[0]) # Va afișa primul element: Minecraft  
print(jocuri[-1]) # Va afișa ultimul element: FIFA
```

Crearea unei liste goale (Empty List):

Uneori, la începutul unui program, nu știm exact ce elemente vom avea. O listă goală se creează pur și simplu punând două paranteze pătrate fără nimic între ele. Este precum o locomotivă de care se vor atașa vagoane ulterior.

```
cos_cumparaturi = [] # 0 listă goală, gata să fie populată ulterior
```

3. Indexarea și accesarea elementelor

Elementele dintr-o listă pot fi accesate și extrase individual printr-un identificator numeric, numit **index**.

Regula de aur: Indexarea într-o listă pornește mereu de la **0**!

Sintaxa generală: `nume_lista[index]`

Ne putem gândi la o listă ca la un tren cu vagoane. Primul element sosit primește mereu locul în primul vagon, adică la indexul 0. Următorul element primește loc în vagonul 1, și tot așa.

Exemplu pentru lista `L = [15, 64, 98, 12]`:

- `L[0] = 15` (primul element)
- `L[1] = 64` (al doilea element)
- `L[2] = 98` (al treilea element)
- `L[3] = 12` (al patrulea element)

Indexarea negativă: În Python este permisă indexarea de la finalul șirului spre început (de la dreapta la stânga).

- `L[-1]` ne va da întotdeauna ultimul element din listă (12).
- `L[-2]` ne va da penultimul element din listă (98).

Exercițiul 1: Creează în Python o listă numită culori care să conțină 3 dintre culorile tale preferate (scrise ca text). Scrie apoi o instrucțiune print care să afișeze pe ecran a doua culoare din lista ta.

Exercițiul 2: Creează o listă cu temperaturile înregistrate în ultimele 5 zile: `temp = [22, 24, 19, 21, 25]`.

1. Fără să modifici direct linia în care ai creat lista, înlocuiește a treia temperatură din șir (19) cu 20.
2. Scrie o instrucțiune print care să afișeze rezultatul matematic (scăderea) dintre ultima temperatură din listă și prima temperatură din listă, folosind indicii aferenți.

4. Modificarea listelor (Actualizare, Adăugare, Ștergere)

A. Citirea datelor

```
sporturi = []
for i in range(3):
    sport_nou = input("Introdu un sport: ")
    sporturi.append(sport_nou)

print("Lista finală este:", sporturi)
```

B. Modificarea unui element (Schimbăm încărcătura unui vagon)

```
jocuri[1] = "Valorant"
print(jocuri)
# Roblox a fost înlocuit! Lista devine: ["Minecraft", "Valorant", "FIFA"]
```

C. Adăugarea elementelor noi

- `lista.append(element)`: Adaugă un element nou la finalul listei.

```
jocuri = ["Minecraft", "FIFA"]
jocuri.append("Tetris")
print(jocuri)
# Rezultat: ['Minecraft', 'FIFA', 'Tetris']
```

D. Inserarea elementelor noi

- `lista.insert(index, element)`: Inserează un element exact la indexul specificat, împingând celelalte elemente mai la dreapta.

```
jocuri = ["Minecraft", "FIFA"]
# Inserăm "Roblox" pe poziția 1 (între Minecraft și FIFA)
jocuri.insert(1, "Roblox")
print(jocuri)
# Rezultat: ['Minecraft', 'Roblox', 'FIFA']
```

E. Ștergerea elementelor

- `lista.remove(element)`: Șterge prima apariție a valorii specificate din listă.

```
jocuri = ["Minecraft", "Roblox", "FIFA", "Minecraft"]
jocuri.remove("Minecraft")
print(jocuri)
# Rezultat: ['Roblox', 'FIFA', 'Minecraft'] (A șters doar primul "Minecraft" pe care l-a găsit)
```

- `lista.pop()`: Elimină și îți returnează ultimul element din listă. Dacă specificăm un index, ex. `lista.pop(0)`, va șterge primul element.

```
jocuri = ["Roblox", "FIFA", "Tetris"]
# Ștergem ultimul element
joc_sters = jocuri.pop()
print("Am șters:", joc_sters) # Rezultat: Am șters: Tetris
print(jocuri)                # Rezultat: ['Roblox', 'FIFA']
# Ștergem elementul de la indexul 0
jocuri.pop(0)
print(jocuri)                # Rezultat: ['FIFA']
```

Exercițiul 2: Ai următoarea listă de inventar a magazinului tău: `inventar = ["apa", "paine", "lapte"]`.

1. Un client tocmai a cumpărat pâinea. Folosește o funcție pentru a șterge "paine" din listă.
2. A sosit marfă nouă! Adaugă "ciocolata" la finalul listei.
3. Afișează pe ecran lista actualizată a magazinului tău.

5. Necesitatea listelor și Parcurgerea lor (Iterarea)

Când și de ce avem nevoie de liste?

Avem următoarea problemă:

Se dau mediile unui elev la 3 discipline (numere naturale). Aflați media generală a elevului.

Rezolvare:

```
med1 = int(input("media1 = "))
med2 = int(input("media2 = "))
med3 = int(input("media3 = "))
media_generala = (med1 + med2 + med3) / 3
```

Dar dacă problema ne cere să lucrăm cu mediile a 25 de elevi? Declararea a 25 de variabile independente este inefficientă. O listă vine în ajutor, permițându-ne să stocăm toate aceste 25 de valori într-o singură structură (`medii_generale = []`) și să le accesăm rapid.

În practică, numărul de elevi dintr-o clasă variază. De aceea, cel mai elegant și corect este să generalizăm problema: lăsăm utilizatorul să introducă de la tastatură numărul de elevi (pe care îl vom nota cu n)

Rezolvare:

```
# Citim numărul total de elevi
n = int(input("Introdu numărul de elevi din clasă: "))

# Creăm lista goală
medii_generale = []

#preluăm datele fiecărui elev din clasă, pentru a-I calcula media și a o adăuga în lista mediilor
for i in range(n):
    print(f"--- Date pentru elevul {i + 1} ---")
    med1 = int(input("media 1 = "))
    med2 = int(input("media 2 = "))
    med3 = int(input("media 3 = "))

    # Calculăm media curentă
    media_curenta = (med1 + med2 + med3) / 3

    # Adăugăm media calculată la finalul listei
    medii_generale.append(media_curenta)

print("\nMediile clasei sunt:", medii_generale)
```

Parcurea / Iterarea listelor

Iterarea înseamnă a accesa pe rând fiecare element din listă pentru a executa o operație cu el. În Python, realizăm acest lucru prin intermediul unei structuri repetitive:

- **Metoda 1: Parcurea directă a elementelor** (Când ne interesează doar valoarea, nu și poziția ei).

```
fructe = ["Mar", "Banana", "Cireasa"]
for fruct in fructe:
    print(f"Am mancat un {fruct}")
```

- **Metoda 2: Parcurea prin index** (Când avem nevoie și de poziția/indexul elementului). Se utilizează funcțiile `range()` și `len()`.

```
elevi = ["Ana", "Mihai", "Elena"]
for i in range(len(elevi)):
    print(f"Elevul de la banca {i} este {elevi[i]}")
```

Exercițiul 4: Ai o listă de nume: `nume = ["Alex", "Maria", "Ionut", "Daria"]`.

1. Folosește metoda potrivită pentru a afișa lungimea listei.
2. Iterează lista pentru a afișa textul `"Salut, [nume]!"` pentru fiecare persoană.